

# RATE-SENTINEL LOW LEVEL DESIGN (LLD)

Version: 1.0

Date: 2026-07-16

Project: rate-sentinel

## 1. SCOPE

This LLD defines module-level and class-level design for key runtime capabilities:

- LLD-1: Rate limiting decision engine
- LLD-2: Authentication and security filter chain
- LLD-3: OTP generation and verification
- LLD-4: Idempotent payment processing
- LLD-5: Notification dispatch over Kafka
- LLD-6: Data model and persistence behavior

## 2. LLD-1: RATE LIMITING DECISION ENGINE

Primary classes:

- 'filter/RateLimitFilter.java'
- 'service/RateLimitService.java'
- 'algorithm/RateLimitAlgorithmFactory.java'
- 'algorithm/SlidingWindowAlgorithm.java'
- 'algorithm/TokenBucketAlgorithm.java'
- 'algorithm/FixedWindowAlgorithm.java'

### 2.1 RESPONSIBILITIES

- 'RateLimitFilter'
  - Resolve 'clientId' (request attribute from JWT or IP fallback).
  - Skip exempt routes ('/actuator', '/api/auth', '/favicon.ico').
  - Invoke service and write 'X-RateLimit-\*' headers.
  - Return 429 JSON body for denied requests.
- 'RateLimitService'
  - Resolve best matching rule using priority and pattern matching.
  - Use cache for rule lookup and fallback defaults when needed.
  - Delegate quota checks to selected algorithm strategy.
- Algorithm implementations
  - 'SlidingWindowAlgorithm': high-accuracy windowing using Redis ZSET and atomic Lua script.
  - 'TokenBucketAlgorithm': burst-support with refill calculations.
  - 'FixedWindowAlgorithm': low-overhead counter per bucket.

### 2.2 RULE SELECTION LOGIC

Rule resolution order:

1. Exact ('clientId', 'route')
2. Wildcard client ('\*', 'route')

3. Pattern route match among active rules for client or wildcard client
4. Global ('\*', '\*')
5. Hardcoded default from properties

## 2.3 RUNTIME SEQUENCE

1. 'RateLimitFilter#doFilterInternal'
2. 'RateLimitService#checkRateLimit'
3. 'RateLimitService#getRule'
4. 'RateLimitAlgorithmFactory#getAlgorithm'
5. 'RateLimitAlgorithm#isAllowed' and '#getRemaining'
6. Filter writes headers and either stops (429) or forwards chain

## 3. LLD-2: AUTHENTICATION AND SECURITY

Primary classes:

- 'filter/JWTAuthFilter.java'
- 'security/JWTTokenProvider.java'
- 'config/SecurityConfig.java'
- 'config/UserConfig.java'
- 'controller/AuthController.java'

### 3.1 FILTER CHAIN POSITIONING

- 'JWTAuthFilter' is placed before 'UsernamePasswordAuthenticationFilter'.
- It parses Bearer token, validates signature/expiry, and stores principal in security context.
- It sets 'request.setAttribute("clientId", subject)' for downstream rate limiting.

### 3.2 ACCESS MODEL

- Public: '/api/auth/\*\*', '/actuator/\*\*'
- Authenticated: all other endpoints
- Admin-specific authorization enforced by route and role-based logic where configured.

### 3.3 LOGIN FLOW

1. 'AuthController' accepts credentials.
2. 'AuthenticationManager' authenticates principal.
3. 'JWTTokenProvider' creates signed token with role claim and expiration.
4. Token is returned to caller for subsequent protected calls.

## 4. LLD-3: OTP GENERATION AND VERIFICATION

Primary classes:

- 'service/OTPService.java'
- 'controller/OTPController.java'
- 'repository/OTPRepository.java'
- 'model/OTPRecord.java'

### 4.1 GENERATE OTP ('GENERATESOTP')

1. Check lock key 'otp:lock:{identifier}' in Redis.
2. Generate random numeric OTP ('SecureRandom') with configured length.
3. Hash OTP via BCrypt.
4. Persist OTP record with expiry timestamp.
5. Build and dispatch 'NotificationEvent'.
6. Return success response.

#### 4.2 VERIFY OTP ('VERIFIESOTP')

1. Check lock status.
2. Fetch latest unused OTP record for identifier/channel.
3. Validate expiry.
4. Increment attempts and evaluate max-attempt threshold.
5. On threshold breach, create lock key with configured lockout TTL.
6. Compare input OTP against BCrypt hash.
7. If matched, mark record as used and return true; else return false.

#### 4.3 ERROR CONDITIONS

- No active OTP record found.
- OTP expired.
- Account currently locked due to excessive attempts.

### 5. LLD-4: IDEMPOTENT PAYMENT PROCESSING

Primary classes:

- 'service/PaymentService.java'
- 'controller/PaymentController.java'
- 'repository/PaymentRepository.java'
- 'model/Payment.java'

#### 5.1 PROCESSING STEPS ('PROCESSPAYMENT')

1. Build Redis key 'payment:idem:{idempotencyKey}'.
2. If Redis key exists, return existing payment by idempotency key.
3. Else check durable existence in MySQL repository.
4. If absent, create 'Payment' as 'PENDING' and persist.
5. Execute gateway operation (currently internal method placeholder).
6. On success: set 'SUCCESS', set 'processedAt', persist, cache key in Redis with TTL.
7. On failure: set 'FAILED', persist failure reason.
8. Dispatch corresponding notification template.

#### 5.2 CONSISTENCY NOTES

- DB unique constraint on idempotency key prevents duplicate inserts.
- Redis accelerates retries while DB remains source of truth.
- Event dispatch is asynchronous and does not block payment API response.

### 6. LLD-5: NOTIFICATION DISPATCH

Primary classes:

- 'service/NotificationDispatcherService.java'
- 'config/KafkaConfig.java'
- 'model/NotificationEvent.java'
- 'enums/NotificationChannel.java'

## 6.1 TOPIC ROUTING

- Channel-to-topic mapping is resolved using application properties.
- Kafka producer uses String key serializer and JSON value serializer.
- Topics are provisioned through 'NewTopic' beans during startup.

## 6.2 DISPATCH CONTRACT

'NotificationEvent' payload fields:

- eventId, clientId, recipient
- channel, templateId, templateParams
- priority, correlationId

Dispatch behavior:

- Service logs enqueue outcome.
- Failures are logged; API workflows remain non-blocking.

## 7. LLD-6: PERSISTENCE AND CACHING DESIGN

Primary entities and repositories:

- 'RateLimitRule' <-> 'RateLimitRuleRepository'
- 'OTPRecord' <-> 'OTPRepository'
- 'Payment' <-> 'PaymentRepository'

### 7.1 MYSQL PERSISTENCE

- JPA/Hibernate with schema auto-update ('spring.jpa.hibernate.ddl-auto=update').
- Rule table stores algorithm and quota parameters.
- OTP table stores hashed OTP with status and attempts.
- Payment table stores idempotency lifecycle.

### 7.2 REDIS USAGE MATRIX

- Counters/state for rate-limiter algorithms.
- OTP lockout keys and timeout control.
- Idempotency cache for payment retries.
- Rule lookup cache with TTL and eviction on admin updates.

## 8. CONFIGURATION AND OPERATIONAL PARAMETERS

From 'application.properties':

- OTP: length, expiry, max attempts, logout
- JWT: secret, expiration
- Rate-limiter defaults: limit, window, algorithm
- Redis and Kafka connectivity

Operational guidance:

- Keep JWT secret externalized per environment.
- Keep Redis timeouts and pool settings aligned with expected traffic.
- Tune topic partitions based on event throughput.

## 9. KNOWN CONSTRAINTS AND DESIGN CONSIDERATIONS

- Current payment gateway integration method is a placeholder; production adapter needed.
- Security config currently permits all authenticated traffic beyond explicit public routes.
- Endpoint naming conventions are mixed ('/api/v1/payment/createPayment'); standardization recommended.
- LLD assumes single-region deployment with shared Redis/Kafka.